

UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE CIENCIAS, 2027-I

MODELADO Y PROGRAMACIÓN

PROYECTO: CHAT MULTIUSUARIO

ALUMNO: Valeria Becerril Acosta

NÚMERO DE CUENTA: 323128489

28 de septiembre de 2026

Resumen

Este proyecto implementa un chat multiusuario en C++ usando el patrón de diseño modelo vista controlador y apoyándose de herramientas como Meson para la construcción, Doxygen para la documentación, además de una biblioteca JSON para la creación de mensajes.

Índice

Introducción	3
Problema	3
Análisis	3
Diseño	6
Implementación	9
Conclusión	9

Introducción

Este Proyecto busca implementar el cliente y el servidor de un chat multiusuario funcional mediante la implementación de ciertos patrones de diseño (como el MVC) que faciliten su desarrollo. A su vez, se espera seguir el enfoque de gestión de proyectos usando el modelo de la cascada, así como prácticas como el código limpio, que no solo facilitan el desarrollo de proyectos, sino que aseguran una mejor comprensión de cualquier proceso que lo implemente.

Problema

Al ser un chat multiusuario, como el nombre del proyecto nombre lo indica, se espera que varias personas se conecten al servidor de manera simultánea sin generar problemas tanto para el cliente como para el servidor. En esencia, el servidor debe permanecer siempre activo, ningún error debe terminar el programa, y en caso de que alguno se detecte, el servidor debe ser capaz de manejarlo sin que afecte a ninguno de los usuarios.

Para llevar esto a cabo se necesita del uso de programación concurrente, esto para poder realizar diferentes tareas de manera simultánea, como sería responder las diferentes solicitudes de los clientes.

También es necesaria una comunicación mediante sockets entre el cliente y el servidor. Los usuarios no pueden comunicarse directamente entre sí; en su lugar, necesitan enviar una solicitud al servidor para que este la procese y genere una respuesta. Este proceso se aplica a cualquier acción que el usuario desee realizar. Primero tiene que pasar por el servidor y, dependiendo de su validez, se realizará o se desconectará al cliente.

Análisis

A grandes rasgos, este chat se puede dividir en el cliente y el servidor, simplemente explicando las acciones que realizan y cómo interactúan entre sí a través de los mensajes que se envían.

- **Identificación del usuario**

- **Petición del cliente:** Solicitud para conectarse e identificarse con un nombre de usuario.
- **Respuesta del servidor:** Si el nombre está disponible, se envía un mensaje al resto de los clientes informando del nuevo usuario, en caso contrario responderá que el nombre de usuario ya está en uso.

- **Cambiar estado**

- **Petición del cliente:** Cambiar el estado del usuario entre **ACTIVE**, **AWAY** y **BUSY**.
- **Respuesta del servidor:** Si el nuevo estado es diferente al anterior, el servidor informa a los demás clientes del cambio.

- **Lista de usuarios**

- **Petición del cliente:** Solicita la lista de los usuarios en el chat.
- **Respuesta del servidor:** Regresa un objeto con los nombres y estados de los usuarios.

- **Mensaje privado**

- **Petición del cliente:** Mandar un mensaje privado a un usuario.
- **Respuesta del servidor:** Si el usuario existe el mensaje se envía, en caso contrario responderá que el usuario no existe.

- **Mensaje público**

- **Petición del cliente:** Mandar un mensaje público a todos los usuarios disponibles.
- **Respuesta del servidor:** No responde y envía el mensaje a todos los usuarios.

- **Crear sala de chat**

- **Petición del cliente:** Crear una nueva sala de chat con un nombre.
- **Respuesta del servidor:** Si el nombre está disponible, se creará una sala con solo el usuario en ella, en caso contrario responderá que la sala ya existe.

- **Invitar usuarios**

- **Petición del cliente:** Invitar a uno o más usuarios a una sala a la que pertenece el usuario.
- **Respuesta del servidor:** Si la sala y los usuarios invitados existen, el servidor no responde e invita a cada usuario, en caso contrario responderá que la sala (o el usuario) no existe.

- **Unirse a una sala**

- **Petición del cliente:** Unirse a una sala a la que el usuario ha sido invitado.
- **Respuesta del servidor:** Si la sala existe y el usuario fue invitado, el usuario se unirá a la sala, en caso contrario el servidor responderá que el cuarto no existe o que el usuario no fue invitado, según sea el caso.

- **Usuarios en la sala**

- **Petición del cliente:** Solicita un diccionario con los usuarios de la sala.
- **Respuesta del servidor:** Si la sala existe y el usuario pertenece a ella, el servidor regresará un diccionario con los nombres y estados de los usuarios. Si la sala existe pero el usuario no ha sido invitado o no se ha unido, el servidor responderá que el usuario no se ha unido a la sala y, si la sala no existe, el servidor respondera indicando que no existe.

- **Mensaje a una sala**

- **Petición del cliente:** Mandar un mensaje a una sala
- **Respuesta del servidor:** Si la sala existe y el usuario pertenece a ella, el servidor no responderá y envía el mensaje a todos los usuarios de la sala. Si la sala existe pero el usuario no ha sido invitado o no se ha unido, el servidor responderá que el usuario no se ha unido a la sala y, si la sala no existe, el servidor respondera indicando que no existe.

- **Abandonar una sala**

- **Petición del cliente:** Abandonar una sala a la que pertenece el usuario.
- **Respuesta del servidor:** Si la sala existe y el usuario pertenece a ella, el servidor no responderá y el usuario abandonara la sala. Si la sala existe pero el usuario no ha sido invitado o no se ha unido, el servidor responderá que el usuario no se ha unido a la sala y si la sala no existe el servidor respondera indicando que no existe.

- **Desconectarse**

- **Petición del cliente:** Desconectar al usuario del chat y abandonar todas las salas a las que pertenece.
- **Respuesta del servidor:** No responde y el usuario se desconecta. Si este pertenece a alguna sala, las abandona.

Diseño

La implementación se divide principalmente en dos clases: el cliente y el servidor. Ambas se encargan de la comunicación mediante sockets apoyandose de diversas clases, enumeraciones, estructuras y espacios de nombres que se encargan de leer y construir los mensajes que se envían.

La primera versión del chat se baso en este [repositorio](#) y posteriormente se extendio para separar las responsabilidades.

Antes de iniciar el cliente, el programa recibe 3 banderas: ‘-u’ para el nombre de usuario, ‘-p’ para el puerto y ‘-i’ para la ip. Validamos todos los argumentos los utilizando el namespace **Argumentos**. Si todos son válidos entonces los guardamos en la estructura **Parametros** que se usara para inicializar el cliente.

El **Cliente** se encarga de realizar la comunicación entre el usuario y el servidor. Para ello utiliza el namespace **Controlador** que cuenta con dos funciones principales. La primera recibe el comando y los parámetros de la acción que desea realizar el usuario y regresa la solicitud a enviar al servidor. La segunda recibe el mensaje de respuesta del servidor a la solicitud para imprimirlo mediante la **Vista**.

Como propiedades tiene un usuario, que guarda los datos del mismo, el puerto y la ip del servidor y el socket mediante el que se comunica con el

servidor.

Class	
Cliente	
Responsabilidades • Ejecuta el cliente. • Recibe mensajes del servidor. • Lee la entrada del usuario. • Hace solicitudes al servidor. • Desconecta el cliente.	Colaboradores • «namespace» Controlador • «namespace» Vista • Usuario

El **Usuario** es la clase propiedad del cliente que se encarga de guardar la información del usuario como son el nombre, el **EstadoConexion** (**ACTIVE** por defecto) y el color, así como un booleano que nos indica si el mismo todavía esta conectado al servidor.

Class	
Usuario	
Responsabilidades • Define los datos del usuario. • Regresa los datos del usuario. • Modifica el estado de la conexión del usuario.	Colaboradores • «enumeration» Color • «enumeration» EstadoConexion

El **Servidor** se encarga de leer y responder a las solicitudes del cliente. Para ello utiliza la función principal del namespace **Controlador** (diferente al controlador del cliente) que se encarga de procesar los mensajes de los clientes y regresar la estructura **Resultado** que contiene el mensaje de respuesta junto a sus destinatarios, lo que facilita el envío de respuestas a los clientes a comparación de si solo regresara el mensaje. De igual manera, la **Vista** se encarga de imprimir las solicitudes y los mensajes de respuesta.

Como propiedades tiene una estructura **Configuracion** que como su nombre indica, guarda la configuración del servidor establecida al iniciar el programa, el socket del servidor, un diccionario de conexiones que guarda las conexiones activas, un vector con los hilos de ejecución de cada conexion, el mutex para la concurrencia, y un diccionario con las salas activas.

Class Servidor	
Responsabilidades • Ejecuta el servidor. • Lee las solicitudes de las conexiones. • Obtiene las respuestas a las solicitudes. • Envía los mensajes de respuesta a las conexiones. • Desconecta el servidor.	Colaboradores • «struct» Configuracion • «namespace» Controlador • «namespace» Vista • «struct» Resultado • Conexion • Sala

La **Conexion** es el objeto que se crea por cada cliente que se conecta al servidor para mantener la información relevante para comunicarse. Se encarga de guardar el nombre de usuario, el socket y el **EstadoConexion**, así como un número que indica el orden en que se conectó el usuario al servidor.

Class Conexion	
Responsabilidades • Define los datos de la conexión. • Regresa los datos de la conexión. • Desconecta el socket de la conexión.	Colaboradores • «enumeration» EstadoConexion

La **Sala** es el objeto que se crea cada vez que un cliente solicita crear una nueva sala. Esta guarda un diccionario con sus integrantes (conexiones) y un vector con los invitados.

Class Sala	
Responsabilidades • Define los datos de la sala. • Regresa a los invitados y/o integrantes de la sala. • Modifica a los invitados y/o integrantes de la sala.	Colaboradores • Conexion

Además de las clases mencionadas también se necesita de enumeraciones para la lectura y procesamiento de los mensajes. Las enumeraciones `MensajeCliente` y `MensajeServidor` estan para definir los tipos de mensajes que se pueden enviar entre si el cliente y el servidor. Ambas se apoyan del namespace `Mensaje` para hacer la conversión entre enum, string y JSON.

Implementación

Para la implementación de este proyecto se usó Meson (con Ninja) como sistema de construcción y se crearon varios archivos `meson.build`, uno en cada directorio principal para gestionar los ejecutables, los archivos y las dependencias. Entre estas últimas se encuentra la biblioteca `nlohmann` de C++ para manipular datos JSON. Igualmente se necesitó de Doxygen para la generación de documentación. Esta herramienta no es necesaria para la ejecución del programa, pero si sugerida.

Para correr el programa tenemos dos ejecutables, uno para el cliente y otro para el servidor:

```
./build/src/main/server/servidor 1234
```

Donde el servidor recibe el número del puerto.

```
./build/src/main/client/cliente -p 1234 -u alice -i 127.0.0.1
```

Y el cliente recibe el puerto `-p`, el usuario `-u` y la ip `-i`.

Conclusión

Durante el desarrollo de este proyecto se implementó un sistema de chat multiusuario compuesto por un cliente y un servidor, utilizando sockets para la comunicación y programación concurrente para las múltiples conexiones. Se hizo uso del patrón MVC para separar la comunicación, el procesamiento de solicitudes y la visualización de los mensajes. Asimismo, se utilizaron estructuras y enumeraciones para representar el estado del sistema y los diferentes tipos de mensajes tanto del cliente como del servidor. Finalmente, el proyecto permitió aplicar conceptos de programación orientada a objetos, comunicación mediante sockets, concurrencia y diseño de software.